CLAUDE CODE

TO'LIQ PROFESSIONAL QO'LLANMA

Vibe Coding dan Agentic Engineering gacha — Haqiqiy loyihalarda kunlik foydalanish uchun

O'rnatish va Sozlash	Skills va Agentlar	MCP, LSP, CI/CD
CLAUDE.md & Rules	Hooks & Commands	Enterprise & Teams

2025-2026 | Claude Code v2.x | Java • JavaScript • TypeScript

MUNDARIJA

#	Bo'lim	Mavzu
1	Kirish: Claude Code Nima?	O'rnatish, autentifikatsiya, birinchi sessiya
2	Professional Promptlash	Yaxshi/yomon promptlar, /plan, /compact, kontekst
3	Loyiha Xotirasi	CLAUDE.md, Rules, Memory ierarxiyasi
4	Tizimli Ish Oqimlari	Task lists, /model, effort darajasi, git workflow
5	Domain Bilimlari — Skills	Skill yaratish, frontmatter, chaqirish usullari
6	Agentli Muhandislik	Agents, Subagents, Commands, memory
7	Hooks — Lifecycle Avtomatizatsiya	PreToolUse, PostToolUse, Stop, SessionStart
8	MCP Serverlar	GitHub, Playwright, DB, Slack, mem0
9	LSP Serverlar	11 til, plugin marketplace, cclsp, LSP+Grep
10	CI/CD va Headless Rejim	GitHub Actions, Docker, skript rejimi, JSON output
11	Enterprise va Jamoalar	managed-settings, SSO, parallel agentlar
12	Haqiqiy Stsenariylar	Bug fix, Code review, Feature, Refactoring oqimi
13	Barcha Slash Buyruqlar	To'liq ma'lumotnoma
14	IDE Integratsiya va Xarajat Optimallashtirish	VS Code, IntelliJ, model tanlash strategiyasi
15	Oltin Qoidalar va Cheat Sheet	Professional foydalanish tamoyillari, resources
16	Real Loyiha: Java+Spring+Vue 3+TS	Qadamma-qadam full stack loyiha qurilishi

Claude Code — Anthropic tomonidan yaratilgan, terminal orqali ishlaydigan **agentic AI coding assistant**. **U oddiy chatbot emas** — loyihangizni o'qiydi, fayllarni tahrirlaydi, shell buyruqlarini bajaradi va haqiqiy dasturchi sifatida ishlaydi.

Vibe Coding vs. Professional Foydalanish

Vibe Coding (0%)	Professional Foydalanish (100%)
> add a notes feature	Loyiha arxitekturasi to'liq tanish, pattern'lar hurmat qilinadi
Random schema yaratiladi	Mavjud model/schema'ga mos kod yaratiladi
Frontend: standalone sahifa, nav yo'q	Sidebar'ga avtomatik nav qo'shiladi
Testlar yo'q	Mavjud test uslubida avtomatik testlar yoziladi
Har yangi feature — lotereya	Barqaror, production-ready, testlangan kod

1.1 O'rnatish

npm orqali (barcha platformalar — tavsiya etilgan)

```
npm install -g @anthropic-ai/claude-code

claude --version    # versiyani tekshirish
npm update -g @anthropic-ai/claude-code  # yangilash
```

Homebrew orqali (macOS)

```
brew install claude-code
brew upgrade claude-code
```

ESLATMA

O'rnatgandan keyin /doctor buyrug'ini ishga tushiring — barcha komponentlarni tekshiradi.

Tizim Talablari

Komponent	Tavsif
Node.js 18+	Claude Code npm package sifatida tarqatiladi
macOS / Linux / Windows WSL	Barcha platformalarda ishlaydi
Terminal	iTerm2, Alacritty, Windows Terminal tavsiya etiladi
Internet	Anthropic API ga kirish uchun

1.2 Autentifikatsiya

A — Claude.ai Obuna (OAuth)

```
claude
# Login promptida 'Claude.ai' ni tanlang -> Brauzer ochiladi
```

Tarif	Narx	Haftalik Limit	Sonnet 4.5	Opus 4.6
Pro	$20/oy	Asosiy limit	Ha	Yo'q
Max 5x	$100/oy	5x Pro	Ha	Ha
Max 20x	$200/oy	20x Pro	Ha	Ha

Haftalik limitlar har 7 kunda yangilanadi. Claude web, desktop va Claude Code o'rtasida **umumiy** hisoblanadi.

B — API Key (Token asosida to'lov, haftalik chek yo'q)

```
export ANTHROPIC_API_KEY=sk-ant-...
claude
```

Model	Kirish (1M token)	Chiqish (1M token)	Context
Claude Opus 4.6	$15	$75	200K (1M beta)
Claude Sonnet 4.5	$3	$15	200K (1M beta)
Claude Haiku 4.5	$0.80	$4	200K (1M beta)

1.3 Birinchi Sessiya

```
cd ~/mening-loyiham
claude

# +-----------------------------------------+
# |  Claude Code v2.x                       |
# |  Model: claude-opus-4-6                 |
# |  Project: ~/mening-loyiham              |
# +-----------------------------------------+
# > Promptingizni yozing...
```

Klaviatura	Amal
Enter	Promptni yuborish
Shift+Enter yoki \\	Yangi qator (ko'p qatorli input)
Ctrl+C	Joriy generatsiyani bekor qilish
Ctrl+D yoki /exit	Yopish
Option+P (Mac) / Alt+P	Tezkor model almashtirish
Ctrl+V	Screenshot yuklash
!buyruq	Claude tahlilisiz bash bajarish (masalan: !git status)

2. PROFESSIONAL PROMPTLASH TEXNIKASI

SAVIYA: 0% -> 20% Yaxshi prompt = yaxshi natija

2.1 Yaxshi va Yomon Promptlar — Java/JS/TS Misollari

Java misoli

Yomon Prompt	Professional Prompt
> fix the null error	> src/main/java/com/app/service/UserService.java 84-qatordagi getUserById() metodida NullPointerException tuzat. Optional<User> qaytarsin, mavjud orElseThrow pattern'iga amal qil
> add auth	> Spring Security bilan JWT auth qo'sh. SecurityConfig.java ga amal qil, mavjud UserDetailsService implementatsiyasidan foydalan

TypeScript misoli

Yomon Prompt	Professional Prompt
> add validation	> src/controllers/userController.ts da POST /api/users endpointiga Zod bilan validatsiya qo'sh. @types/zod o'rnatilgan, mavjud productController.ts dagi pattern'ga amal qil
> fix the type error	> src/types/api.ts 42-qatordagi ApiResponse<T> tipidagi 'Property does not exist' xatoni tuzat. Generic type constraint qo'shish kerak

JavaScript misoli

Yomon Prompt	Professional Prompt
> make it async	> routes/todos.js dagi getAllTodos() funksiyasini async/await ga o'tkaz. Mavjud error handling middleware bilan mos bo'lsin, Express 4 uslubida
> add tests	> __tests__/userService.test.js faylini yaratib UserService uchun Jest testlari yoz. Mavjud productService.test.js dagi mock pattern'iga amal qil

2.2 Kontekst Berish — @ Belgisi

@ belgisi bilan fayllarni to'g'ridan-to'g'ri promptga qo'shing:

```
# Java: Mavjud pattern'dan foydalanish
> @src/main/java/com/app/service/ProductService.java va
  @src/main/java/com/app/model/Product.java ni ko'rib,
  OrderService uchun xuddi shunday tuzilma yaratib ber

# TypeScript: Tip xatolarini tuzatish
> @src/types/api.ts va @src/utils/helpers.ts ni ko'rib,
  ApiResponse<T> generic tipidagi xatoni tuzat

# JavaScript: Test yozish
> @tests/productController.test.js ni o'qib,
  orderController.test.js uchun xuddi shu uslubda testlar yoz
```

Kontekst Turi	Qanday Foydalanish
@fayl/yo'li	Aniq faylni Claude ga o'qitish — eng muhim texnika
Screenshot Ctrl+V	UI muammolari, stack trace rasmlari
Error log to'liq	To'liq stack trace ni promptga nusxa ko'chirish
Pattern ref	"ProductService.java dagi pattern'ga amal qil" deyish

Multimodal Input — Screenshot bilan Ishlash Misoli

UI bug'ini tuzatishda: **Ctrl+V** bilan screenshot yuklang, keyin promptni yozing:

```
● ● ●      claude — screenshot yuklash

> [Ctrl+V bilan screenshot joylashtirildi]
  Screenshot qabul qilindi: ui-bug-screenshot.png (1440x900)

> Vue komponentida ko'rsatilgan bu xatoni tuzat.
  @src/components/OrderList.vue faylida izla.
  Tailwind CSS bilan stil muammosi bo'lsa ham ko'rsat.

  Analyzing screenshot...
  Reading src/components/OrderList.vue...

  Topildi: v-for direktivasida :key atributi yetishmayapti.
  OrderList.vue 23-qator: <div v-for="order in orders">
  Tuzatish: <div v-for="order in orders" :key="order.id">
```

↑ *Ctrl+V bilan yuklangan screenshot Claude tahlil qilib, @fayl referens bilan birgalikda ishlatiladi*

2.3 Context Window va /compact

Context window'ni to'ldiradigan narsalar:

- Sizning promptlaringiz va xabarlaringiz
- Claude o'qigan har bir fayl (katta Java fayllari tez to'ldiradi!)
- Bash buyruq natijalari (mvn build, npm test chiqishlari)
- Claude'ning javoblari

MUHIM

Context ~50% to'lganda /compact ni QO'LDA ishga tushiring. Avtomatikni kutmang — nazorat sizda qolsin.

```
# Context'ni siqish (suhbat tarixini xulosa sifatida saqlaydi)
> /compact
```

2.4 /plan — Kod Yozishdan Oldin Rejalashtirish

/plan yo'q	/plan bilan
Kod darhol yoziladi	Claude avval loyihani o'rganadi (read-only rejim)
Noto'g'ri yondashuv — hamma narsa qayta qilinadi	Yondashuv taklif etiladi, siz tasdiqlaysiz
50% context isrof bo'ladi	To'g'ri yondashuv boshidanoq

```
> /plan
> Spring Boot loyihasiga OAuth2 + JWT autentifikatsiya qo'sh
#  Claude bajaradi:
#  1. pom.xml, SecurityConfig.java, mavjud auth komponentlarni o'qiydi
#  2. Reja taklif etadi: qaysi fayllar o'zgaradi, qaysi sinflar qo'shiladi
#  3. Siz tasdiqlaysiz -> bajaradi
```

DIQQAT

2-3 fayldan ortiq yoki arxitektura qarorlarini talab qiladigan BARCHA vazifalar uchun /plan bilan boshlang.

3. LOYIHA XOTIRASI — CLAUDE.MD VA RULES

SAVIYA: 20% -> 40% Claude loyihangizni tanib oladi

3.1 CLAUDE.md — Loyiha Uchun Qo'llanma

CLAUDE.md — Claude har sessiya boshida o'qiydigan markdown fayl. Loyihangizning arxitekturasi, konventsiyalari va qoidalarini tushuntiradi.

```
> /init   # Loyiha tuzilmasini skanlab CLAUDE.md yaratadi
```

CLAUDE.md — Java Spring Boot Loyihasi Uchun Namuna

```
# CLAUDE.md

## Loyiha
E-commerce REST API: Spring Boot 3.2 + PostgreSQL + Redis

## Tuzilma
- src/main/java/com/app/
  - controller/  -- REST endpoint'lar (@RestController)
  - service/     -- Biznes mantiq (@Service)
  - repository/  -- JPA repository'lar
  - model/       -- JPA entity'lar
  - dto/         -- Request/Response DTO'lar
  - config/      -- Spring konfiguratsiya

## Buyruqlar
- mvn spring-boot:run         # ishga tushirish
- mvn test                    # testlar
- mvn clean package           # build

## Konventsiyalar
- Barcha endpoint'lar uchun Swagger @Operation annotatsiya
- Service qatlamida @Transactional
- Global @ControllerAdvice exception handling
- Lombok @Builder, @Data annotatsiyalari
- Test: ProductServiceTest.java uslubiga amal qil
```

CLAUDE.md — TypeScript Node.js Loyihasi Uchun Namuna

```
# CLAUDE.md

## Loyiha
Todo REST API: Express 4 + TypeScript + Prisma ORM + PostgreSQL

## Tuzilma
- src/
  - routes/      -- Express route handler'lar
  - controllers/ -- Request/response logic
  - services/    -- Biznes mantiq
  - middleware/  -- Auth, validation, error handling
  - types/       -- TypeScript type ta'riflari
  - prisma/      -- Prisma schema va migrations

## Buyruqlar
- npm run dev       # ts-node-dev bilan ishga tushirish
- npm test          # Jest testlar
- npx prisma migrate dev  # migration

## Konventsiyalar
- Strict TypeScript: no 'any' types
- Zod bilan request validatsiya
- productController.ts uslubiga amal qil
- Har endpoint uchun unit test majburiy
```

3.2 Nima Kiritish / Kiritmaslik

KIRITISH KERAK	KIRITMASLIK KERAK
Loyiha arxitekturasi umumiy ko'rinishi	Butun API dokumentatsiyasi / Javadoc
Build va test buyruqlari	Uzun kod namunalari
Muhim fayl joylashuvi	Secrets, API kalitlar, parollar
Kodlash konventsiyalari (annotation, pattern)	Tez-tez o'zgaruvchan ma'lumotlar
Keng tarqalgan pattern'lar (qaysi sinfga rioya qilish)	150 qatordan ortiq matn

DIQQAT

CLAUDE.md ni 150 qatordan qisqa saqlang! Uzun fayllar ko'rsatmalarni suyultiradi — Claude ularning bir qismini e'tiborsiz qoldirishi mumkin.

3.3 Memory Ierarxiyasi

```
# Sessiya boshida yuklanish tartibi:
~/.claude/CLAUDE.md           # Global -- barcha loyihalar
.claude/CLAUDE.md             # Loyiha -- git orqali jamoaga ulashiladi
.claude/CLAUDE.local.md       # Shaxsiy -- git'ga kirmaydi
# Subpapka-specific ko'rsatmalar:
src/main/java/CLAUDE.md       # Faqat Java kodi uchun
src/test/CLAUDE.md            # Faqat testlar uchun
```

```
> /memory   # Xotira fayllarini redaktorda ochish
```

3.4 Rules — Yo'l-Scoped Ko'rsatmalar

Rules — aniq papka yo'llariga (glob) bog'liq holda faollashadigan alohida markdown fayllar. Faqat tegishli kontekstda yuklanadi.

Java Test Rules Namunasi

```
# .claude/rules/java-tests.md
---
globs: ["src/test/**/*.java"]
---
Java testlar yozganda:
- JUnit 5 va Mockito ishlatilsin
- ProductServiceTest.java uslubiga amal qiling:
  - @ExtendWith(MockitoExtension.class)
  - @Mock va @InjectMocks annotatsiyalar
  - given/when/then tuzilmasi
  - AssertJ assertThat() ishlatilsin
- Nom: test{Amal}{Stsenary}_{Kutilgan natija}()
- Har method uchun: success + edge case + failure testlari
```

TypeScript Rules Namunasi

```
# .claude/rules/ts-api.md
---
globs: ["src/controllers/**/*.ts", "src/routes/**/*.ts"]
---
TypeScript API endpointlar yozganda:
- productController.ts dagi tuzilmaga rioya qiling
- Barcha request bodylarni Zod bilan validatsiya qiling
- try/catch bilan global error handler'ga uzating
- Response: { success: boolean, data: T, message?: string }
- HTTP status kodlarini to'g'ri ishlating (201, 400, 404, 500)
```

IERARXIYA
CLAUDE.md (doim, <150 qator) -> Rules (yo'l-scoped, avtomatik) -> Skills (talab bo'yicha)

4. TIZIMLI ISH OQIMLARI

SAVIYA: 40% -> 50% Tizimli yondashuvlar isrof harakatni oldini oladi

4.1 Vazifalar Ro'yxati (Task Lists)

Murakkab vazifalar uchun Claude progress'ni kuzatib boradigan task list tuzadi:

```
# Java misoli:
> Spring Boot loyihasiga foydalanuvchi autentifikatsiyasini qo'sh:
   - User entity va UserRepository (JPA)
   - UserService: register, login metodlari
   - JWT token generation va validation (jjwt library)
   - SecurityConfig: /auth/** public, qolganlar protected
   - AuthController: POST /api/auth/register, /api/auth/login
   - JUnit testlari har service va controller uchun

# TypeScript misoli:
> Express loyihasiga foydalanuvchi autentifikatsiyasini qo'sh:
   - User Prisma model (schema.prisma)
   - authService.ts: register, login, verifyToken
   - JWT middleware (authMiddleware.ts)
   - authController.ts: POST /api/auth/register, /api/auth/login
   - Zod schemas: RegisterSchema, LoginSchema
   - Jest testlari authService va authController uchun
```

MUHIM

Har kichik vazifa 50% context ichida tugallanishi kerak. Har kichik vazifadan keyin commit qiling.

4.2 /model — Model Tanlash va Effort Darajasi

Model	Qachon Ishlatish
Claude Opus 4.6	Murakkab: Spring Security konfiguratsiya, arxitektura qarorlari, katta refactoring
Claude Sonnet 4.5	Kunlik: yangi endpoint, DTO qo'shish, xatoni tuzatish
Claude Haiku 4.5	Oddiy: faylni o'qib tushuntirish, qisqa savollar

```
> /model                    # Model pickerni ochish
Option+P (Mac) / Alt+P    # Tezkor model almashtirish
> /fast                     # Opus bilan tezroq output

# Opus Effort darajasi (/model -> Opus -> <- -> strelkalar):
# High:   murakkab arxitektura, nozik xatolar, katta refactoring
# Medium: kunlik kodlash vazifalari
# Low:    tezlik muhim bo'lgan oddiy, aniq vazifalar
```

4.3 Git Workflow

```
> Bu o'zgartishlarni tavsiflovchi xabar bilan commit qil

# Claude bajaradi:
# 1. git status va git diff -- barcha o'zgarishlarni ko'rib chiqadi
# 2. Commit xabari yaratadi (Conventional Commits uslubida)
#    Misol: feat(auth): add JWT token validation in UserService
#    Misol: fix(api): handle null user in getUserById endpoint
# 3. Aniq fayllarni stage qiladi (git add -A EMAS)
```

Ish Oqimi Best Practices

- **Doim /plan bilan boshlang** — 2-3 fayldan ortiq yoki arxitektura qarorlari
- **Kichik vazifalar** — har bir qism 50% context ichida tugasin
- **Har kichik vazifadan keyin commit** — toza rollback nuqtalari
- **/compact qo'lda ~50% da** — avtomatikni kutmang
- **Bir vaqtda bitta vazifa** — mega-promptlar o'rniga aniq, kichik so'rovlar

4.4 Output Styles — Claude'ning Javob Uslubi

Claude'ning javob berish uslubini **/config** orqali sozlash mumkin.

Uslub	Qachon Ishlatish	Misol
Explanatory	Yangi texnologiya o'rganayotganda	Spring Security annotatsiyalari nima qilishini tushuntiradi
Learning	Mustaqil o'sishni xohlasangiz	Claude o'zi yozmaydi, yo'naltiradi: 'Endi @Transactional qo'shib ko'ring'
Custom	Jamoa standarti uchun	Faqat kod + testlar, tushuntirmasiz

```
# Output Style o'rnatish:
> /config   # 'Output Style' bolimiga oting

# Custom style misoli (.claude/settings.json):
{
  "outputStyle": {
    "type": "custom",
    "instructions": "Faqat kod yoz, tushuntirma berma.
      Java: Lombok va Swagger annotatsiyalarni doim qosh.
      TypeScript: har doim toliq tip annotatsiyalari bilan."
  }
}
```

MASLAHAT

Yangi jamoa azosi uchun 'Explanatory', tajribali dasturchi uchun 'Custom'. Bir xil CLAUDE.md, turli tajriba uchun moslashtirilgan javoblar.

5. DOMAIN BILIMLARI — SKILLS

SAVIYA: 50% -> 65% Qayta foydalaniladigan bilim bazasi

5.1 Skills Nima?

CLAUDE.md	Skills
Har doim yuklanadi	Faqat kerak bo'lganda yuklanadi
150 qator chegarasi	Ixtiyoriy uzunlik
Loyiha umumiy ko'rinishi	Muayyan soha bilimlari
Har sessiyada context oladi	Context'ni tejaydi

```
.claude/
  skills/
    java-spring-patterns/
      SKILL.md            # Asosiy skill fayli (majburiy)
      exception-handling.md   # Qo'shimcha (ixtiyoriy)
    ts-api-patterns/
      SKILL.md
    js-testing-patterns/
      SKILL.md
```

5.2 Skill Yaratish — Java Namunasi

```
# .claude/skills/java-spring-patterns/SKILL.md
---
name: java-spring-patterns
description: Spring Boot loyihasida Java kodlash pattern'lari
---

# Java Spring Boot Pattern'lari

## Service Layer
- @Service, @Transactional annotatsiyalari majburiy
- ProductService.java uslubiga amal qil
- BusinessException faqat service qatlamida throwlansin

## Repository Pattern
- JpaRepository'dan meros olish
- Custom query: @Query annotatsiya yoki method naming
- Pagination: Pageable parametri qabul qilsin

## DTO Pattern
- Request DTO: @Valid, @NotNull, @Size annotatsiyalar
```

```
- Response DTO: Lombok @Builder
- MapStruct bilan DTO <-> Entity conversion

## Exception Handling
- GlobalExceptionHandler.java (@ControllerAdvice)
- Custom exception: BusinessException extends RuntimeException
- HTTP status: 400 (validation), 404 (not found), 500 (server)
```

TypeScript Skill Namunasi

```
# .claude/skills/ts-api-patterns/SKILL.md
---
name: ts-api-patterns
description: TypeScript Express API pattern'lari va konventsiyalar
---

# TypeScript API Pattern'lari

## Controller Pattern
- productController.ts ni namunaviy sifatida kuzat
- Request: Zod schema bilan validate qil
- Response: { success: boolean, data: T, message?: string }
- Async/await, try/catch, next(error) bilan error handling

## Service Pattern
- Biznes mantiq faqat service'da bo'lsin
- Prisma operatsiyalari service'da
- Custom error classes: AppError extends Error

## Type Definitions
- src/types/index.ts da markazlashtirilgan
- Generic API types: ApiResponse<T>, PaginatedResponse<T>
- Strict mode: no 'any', to'liq tip annotatsiyalari
```

5.3 Skill Frontmatter Maydonlari va Chaqirish

Maydon	Tavsif
name	Skill identifikatori (papka nomi ishlatiladi, agar qoldirilsa)
description	Qachon chaqirish kerak — Claude avtomatik topishi uchun
model	opus | sonnet | haiku — bu skill qaysi modelda ishlaydi
allowed-tools	Skill foydalana oladigan toollar ro'yxatini cheklash
context: fork	Skill alohida subagent kontekstida bajariladi

```
# 1. Qo'lda slash buyrug'i
> /java-spring-patterns

# 2. Avtomatik: Claude description bo'yicha o'zi topadi

# 3. Agent'ga oldindan yuklash:
skills:
  - java-spring-patterns

# 4. Og'ir workflowlar uchun izolyatsiya:
context: fork
```

6. AGENTLI MUHANDISLIK — AGENTS VA COMMANDS

SAVIYA: 65% -> 100% Kodingizni biladigan maxsus agentlar

6.1 Agentlar Nima?

Agentlar — .claude/agents/ papkasidagi markdown fayllar. O'z tools, model, skills va xulq-atvoriga ega maxsus Claude persona'larini belgilaydi.

Foydalanish Usuli	Tavsif
claude --agent java-backend	Asosiy agent sifatida — ushbu agent bilan gaplashish
Task tool orqali	Subagent — izolyatsiyalangan kontekstda avtomatik chaqiriladi
settings.json 'agent' maydoni	Default agent — loyihada har doim ushbu agent ishlatiladi

6.2 Java Backend Agent

```
# .claude/agents/java-backend.md
---
name: java-backend
description: Spring Boot Java backend, REST API ishlab chiqish
tools: Read, Write, Edit, Bash, Glob, Grep
model: sonnet
skills:
  - java-spring-patterns
---
Siz Spring Boot 3.2 loyihasi uchun Java backend muhandisissiz.
ProductService.java va ProductController.java ni namuna sifatida ishlat.
Har yangi endpoint uchun JUnit 5 testi yozish MAJBURIY.
Swagger @Operation annotatsiyasini har doim qo'sh.
@Transactional, Lombok, MapStruct, Spring Validation ishlatilsin.
```

Java Backend Agent BILAN vs SIZ:

Agent yo'q	java-backend Agent bilan
Random naming convention	ProductService.java uslubidagi naming
Exception handling yo'q	GlobalExceptionHandler'ga mos @ControllerAdvice
Test yozilmaydi	Har endpoint uchun JUnit 5 testi avtomatik
Swagger annotatsiyasiz	@Operation, @ApiResponse annotatsiyalar qo'shiladi

6.3 TypeScript Frontend Agent

```
# .claude/agents/ts-frontend.md
---
name: ts-frontend
description: React TypeScript frontend, komponentlar va API integratsiya
tools: Read, Write, Edit, Bash, Glob, Grep
model: sonnet
skills:
  - ts-api-patterns
---
Siz React 18 + TypeScript loyihasi uchun frontend muhandisissiz.
Har doim strict TypeScript: no 'any' types.
API chaqiruvlari faqat src/services/ dan o'tsin.
ProductCard.tsx ni namunaviy komponent sifatida kuzat.
Tailwind CSS + shadcn/ui komponentlar ishlatilsin.
```

6.4 JavaScript Node Agent

```
# .claude/agents/js-node.md
---
name: js-node
description: Node.js Express JavaScript backend ishlab chiqish
tools: Read, Write, Edit, Bash, Glob, Grep
model: sonnet
skills:
  - js-testing-patterns
---
Siz Express.js + JavaScript loyihasi uchun backend muhandisissiz.
Async/await va Promise bilan ishlaysan, callback ishlatma.
routes/productRoutes.js ni namuna sifatida ishlat.
Jest + Supertest bilan har endpoint uchun test yoz.
express-validator bilan request validatsiya qil.
```

6.5 Code Reviewer Agent (Xotira bilan)

```
# .claude/agents/code-reviewer.md
---
name: code-reviewer
description: Java/TypeScript/JavaScript kod sifati tekshiruvi
tools: Read, Grep, Glob
model: sonnet
color: green
memory: user
---
Siz code reviewer siz. Quyidagilarni tekshir:
Java: NullPointerException xavfi, N+1 query, @Transactional yetishmasligi
TypeScript: 'any' tipi, type assertion suiiste'moli, strict null check
```

```
JavaScript: callback hell, unhandled promise rejection, memory leak
Umumiy: SOLID, DRY, security vulnerabilities, performance
Xotirangizda ilgari ko'rgan pattern'lar va muammolarni esda tut.
```

6.6 Commands — Murakkab Workflow'lar

```
# .claude/commands/add-feature.md
---
description: Loyihaga yangi feature qo'shish (Java backend + TS frontend)
model: opus
---

# Yangi Feature Qo'shish

1. Foydalanuvchidan qaysi feature kerakligini so'rang (AskUserQuestion)
2. Java backend uchun java-backend agentini chaqiring:
   Task(subagent_type='java-backend',
        prompt='Entity, Service, Controller, Tests yaratib ber')
3. TS frontend uchun ts-frontend agentini chaqiring:
   Task(subagent_type='ts-frontend',
        prompt='React komponent va API integratsiyasini yaratib ber')
4. Testlarni ishga tushiring: mvn test + npm test
5. code-reviewer agentini chaqirib natijani tekshiring
6. Nima yaratilgani haqida xulosa bering

# Chaqirish: > /add-feature
```

ARXITEKTURA PATTERN: Command -> Agent -> Skills

```
/add-feature  (Command) -- kirish nuqtasi
     |
 java-backend   ts-frontend   js-node  -- Agentlar
     |          |
 java-spring-patterns  ts-api-patterns  -- Skills
```

Commands = kirish. Agents = bajarish. Skills = bilim. Toza ajratish.

6.7 Agent Frontmatter va Xotira

Maydon	Tavsif
name	Agent identifikatori
description	Bu agent nima qiladi — avtomatik tanlov uchun
tools	Read, Write, Edit, Bash, Glob, Grep, WebFetch, WebSearch
model	opus | sonnet | haiku
skills	Yuklash uchun skills ro'yxati
memory	user | project | local — sessiyalar bo'ylab o'rganish
color	UI uchun rang: green, blue, red, orange ...

```
# Xotira joylashuvi:
~/.claude/agent-memory/code-reviewer/
  MEMORY.md                  # Birinchi 200 qator avtomatik yuklanadi
  java-antipatterns.md       # Ko'rgan Java muammolari
  ts-type-issues.md          # TypeScript tip xatolari

# Xotira turi:
# user    -- barcha loyihalarda saqlanadi
# project -- jamoa bilan ulashiladi
# local   -- faqat shaxsiy, git'ga kirmaydi
```

7. HOOKS — LIFECYCLE AVTOMATIZATSIYA

7.1 Hooks Nima?

Hooks — Claude'ning hayot siklining muayyan nuqtalarida ishga tushadigan maxsus skriptlar.

Hook Event	Qachon Ishga Tushadi
PreToolUse	Tool ishlatishidan oldin — tasdiqlash, bloklash mumkin
PostToolUse	Tool ishlatgandan keyin — formatlash, linting
Stop	Vazifa tugaganda — natijalarni tekshirish
SessionStart	Sessiya boshida — kontekstni yuklash
Notification	Claude xabar yuborishga harakat qilganda

ESLATMA

Jami 16 ta hook event mavjud, ulardan 5 tasi bajarilishni to'xtata oladi.

7.2 Hook Konfiguratsiyasi — Java Loyihasi Uchun

```
// .claude/settings.json
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": {"tool_name": "Write", "file_pattern": "**/*.java"},
        "hooks": [{
          "type": "command",
          "command": "mvn checkstyle:check -f pom.xml"
        }]
      }
    ],
    "Stop": [
      {
        "hooks": [{
          "type": "command",
          "command": "mvn test -q && echo 'All tests passed'"
        }]
      }
    ]
  }
}
```

TypeScript / JavaScript Loyihasi Uchun

```
// .claude/settings.json
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": {"tool_name": "Write", "file_pattern": "**/*.ts"},
        "hooks": [{"type": "command", "command": "npx prettier --write
$FILE && npx eslint $FILE"}]
      },
      {
        "matcher": {"tool_name": "Write", "file_pattern": "**/*.js"},
        "hooks": [{"type": "command", "command": "npx prettier --write
$FILE"}]
      }
    ],
    "PreToolUse": [
      {
        "matcher": {"tool_name": "Bash"},
        "hooks": [{"type": "command", "command": "echo 'CMD:
$BASH_COMMAND'"}]
      }
    ]
  }
}
```

Keng Tarqalgan Hook Foydalanish Misollari

- **PostToolUse -> Checkstyle/ESLint:** fayl yozilganda avtomatik linting
- **PostToolUse -> Prettier:** TypeScript/JavaScript formatlash
- **PostToolUse -> mvn spotless:** Java kodni Google Java Format bilan formatlash
- **PreToolUse -> rm tekshiruvi:** zararli buyruqlarni bloklash
- **Stop -> mvn test / npm test:** vazifa tugagandan keyin testlar avtomatik
- **SessionStart -> git pull:** sessiya boshida eng yangi o'zgarishlarni olish

7.3 Voice Hooks — Ovozli Boshqarish

claude-code-voice-hooks — mikrofon orqali ovoz buyruqlari bilan Claude Code boshqarish. Qollar band yoki tezroq ishlash uchun ideal.

O'rnatish

```
# Voice hooks o'rnatish:
git clone https://github.com/shanraisshan/claude-code-voice-hooks
cd claude-code-voice-hooks && npm install

# Tizim talablari:
# macOS: SoX  ->  brew install sox
# Linux: arecord  ->  apt install alsa-utils
# OpenAI Whisper API yoki mahalliy model
export OPENAI_API_KEY=sk-...   # Whisper transcription uchun
```

settings.json ga Ulash

```
// .claude/settings.json
{
  "hooks": {
    "SessionStart": [{
      "hooks": [{
        "type": "command",
        "command": "node /path/to/voice-hooks/start.js"
      }]
    }]
  }
}
```

Ovozli Buyruqlar Misollari

Ovozli Buyruq	Claude Bajariladigan Amal
UserService dagi null xatoni tuzat	NullPointerException'ni topib Optional bilan tuzatadi
Testlar yoz	Joriy faylga mos JUnit/Jest testlari yaratadi
Commit qil	git diff ko'rib Conventional Commits uslubida commit qiladi
Plan rejimiga ot	/plan buyrug'ini bajaradi
Compact qil	/compact buyrug'ini bajaradi

ESLATMA

Voice hooks konsepti: Claude Code SessionStart hookiga mikrofon input skriptini ulash. Whisper API yoki mahalliy model bilan amalga oshiriladi. Har loyiha uchun sozlama farqli bo'lishi mumkin.

8. MCP SERVERLAR — TASHQI TOOLLAR INTEGRATSIYASI

8.1 MCP Nima?

Model Context Protocol (MCP) — Anthropic tomonidan ishlab chiqilgan ochiq standart. Claude'ga tashqi toollar va ma'lumot manbalariga ulanishga imkon beradi.

MCP Server	Imkoniyat
Playwright / Chrome DevTools	Brauzer avtomatizatsiyasi, UI sinovi, console log ko'rish
PostgreSQL / MySQL / SQLite	Ma'lumotlar bazasini to'g'ridan-to'g'ri so'rash
GitHub	Pull request, issue, code review, repo boshqarish
Filesystem	Maxsus papka ruxsatlari bilan fayl tizimi
Slack / Jira	Kanal xabarlari, ticket boshqarish
mem0 / Memory	Sessiyalar bo'ylab doimiy, shaxsiylashtirilgan xotira
Serena	Semantic kod tahlili — Java/TS uchun juda foydali

8.2 MCP Server Qo'shish

```
# Mahalliy server qo'shish
claude mcp add my-server -e ENV_VAR=value -- /yo'l/server

# npm package sifatida
claude mcp add-json my-server '{"command":"npx","args":["my-mcp-server"]}'

# HTTP transport
claude mcp add --transport http my-server https://example.com/mcp

# Scope:
# --scope local   -- faqat siz, joriy loyiha [default]
# --scope project -- .mcp.json ga yoziladi (jamoaga ulashiladi)
# --scope user    -- barcha loyihalarda siz uchun
```

.mcp.json — Java + TS Loyiha Uchun To'liq Namuna

```
// .mcp.json (git'ga kiritiladi)
{
  "mcpServers": {
    "playwright": {
      "command": "npx",
      "args": ["@playwright/mcp@latest"]
    },
    "postgres": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres",
               "postgresql://localhost:5432/mydb"]
    },
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp",
      "headers": {"Authorization": "Bearer ${GITHUB_PAT}"}
    },
    "serena": {
      "command": "uvx",
      "args": ["serena", "--project-path", "."]
    }
  }
}
```

```
> /mcp   # MCP serverlarni ko'rish, tekshirish va boshqarish
```

9. LSP SERVERLAR — KOD INTELLEKT

9.1 LSP Nima va Nima Uchun Kerak?

Language Server Protocol (LSP) — VS Code kabi IDE'larda ishlatiladigan kod intellekt standarti. Claude Code v2.0.74 da (2025-yil dekabr) rasman qo'shildi.

LSP'siz (grep)	LSP bilan
Funksiya ta'rifini topish: ~45 sekund	Funksiya ta'rifini topish: ~50ms (900x tezroq!)
Noto'g'ri mosliklar mumkin	Semantik tushunish — aniq natijalar
Tip ma'lumotisiz	Real vaqt diagnostika va tip tekshiruvi
Faqat matn qidirish	Go to Definition, Find References, Hover Info

9.2 Qo'llab-Quvvatlanadigan 11 Til

Til	LSP Server	O'rnatish
Java	Eclipse JDT LS (jdtls)	brew install jdtls | apt install jdtls
TypeScript/JS	vtsls	npm install -g @vtsls/language-server typescript
JavaScript	vtsls (JS ham)	npm install -g @vtsls/language-server
Python	Pyright	pip install pyright | npm install -g pyright
Go	gopls	go install golang.org/x/tools/gopls@latest
Rust	rust-analyzer	rustup component add rust-analyzer
C/C++	clangd	brew install llvm | apt install clangd
C#	OmniSharp	.NET SDK bilan birga keladi
PHP	Intelephense	npm install -g intelephense

Til	LSP Server	O'rnatish
Ruby	Solargraph	gem install solargraph
HTML/CSS	vscode-ls	npm install -g vscode-langservers-extracted

9.3 LSP O'rnatish — Plugin Marketplace

```
# 1. Plugin marketplace'ni ochish
> /plugin
# Discover tab'da 'lsp' ni qidiring

# 2. Kerakli til pluginini o'rnating
/plugin install jdtls@claude-code-lsps       # Java
/plugin install vtsls@claude-code-lsps        # TypeScript/JS
/plugin install pyright@claude-code-lsps      # Python

# 3. ALOHIDA: Til server binary'sini o'rnating
# Java:
brew install jdtls
# TypeScript/JS:
npm install -g @vtsls/language-server typescript
```

DIQQAT

LSP plugin FAQAT ulanish sozlamasini qiladi. Til server binary'sini alohida o'rnatish SHART! '$PATH da topilmadi' xatosi -- binary o'rnatilmagan.

9.4 cclsp — MCP Orqali LSP (Universal Yondashuv)

cclsp — har qanday LSP-mos til serveri bilan ishlashga imkon beradigan MCP server:

```
# O'rnatish
npx cclsp@latest setup           # interaktiv wizard
npx cclsp@latest setup --user    # global (Windows uchun ham shu)
```

cclsp.json — Java + TypeScript + JavaScript

```
{
  "servers": [
    {
      "extensions": ["java"],
      "command": ["jdtls"],
      "rootDir": "."
```

```
    },
    {
      "extensions": ["ts", "tsx"],
      "command": ["typescript-language-server", "--stdio"],
      "rootDir": "."
    },
    {
      "extensions": ["js", "jsx", "mjs"],
      "command": ["typescript-language-server", "--stdio"],
      "rootDir": "."
    }
  ]
}

// Windows: 'command' bo'limiga cmd /c wrapper qo'shing!
// ["cmd", "/c", "typescript-language-server", "--stdio"]
```

.mcp.json ga cclsp qo'shish

```
{
  "mcpServers": {
    "cclsp": {
      "command": "cclsp",
      "env": {
        "CCLSP_CONFIG_PATH": "/home/user/.config/cclsp/cclsp.json"
      }
    }
  }
}
```

9.5 LSP + Grep Kombinatsiyasi (Optimal Yondashuv)

- **LSP** — ta'rifga o'tish va diagnostika (aniq, ~50ms)
- **Grep** — barcha foydalanishlarni topish va matn qidirish (keng qamrov)

```
# Java misoli -- optimal ish oqimi:
# 1. LSP find_definition  -> UserService.java:84 (aniq joylashuv)
# 2. LSP find_references  -> 7 joyda ishlatilmoqda
# 3. Grep 'void.*User'    -> Klassdagi barcha User metodlari
# 4. Faylni o'qish        -> To'liq implementatsiya + Javadoc

# TypeScript misoli:
# 1. LSP find_definition  -> ProductService.ts:23
# 2. LSP find_references  -> 4 controller'da ishlatilmoqda
# 3. Grep 'async.*Product' -> Barcha async Product metodlari
```

10. CI/CD VA HEADLESS REJIM

10.1 Headless / Skript Rejimi

claude -p buyrug'i bilan Claude Code'ni interaktiv bo'lmagan (headless) rejimda ishlatish mumkin — CI/CD pipeline'lar va avtomatizatsiya uchun ideal.

```
# Asosiy sintaksis:
claude -p "prompt matni" [bayroqlar]

# Foydali bayroqlar:
# --output-format json     # Natijani JSON sifatida qaytaradi
# --output-format text     # Oddiy matn (default)
# --output-format stream-json  # Real-time JSON stream
# --no-permission-prompts  # Ruxsat so'rovlarini o'tkazib yuborish
# --allowedTools <list>    # Faqat ko'rsatilgan toollarni ruxsat berish
# --model <model>          # Model tanlash

# Misol: Java loyiha tekshiruvi
claude -p "src/main/java/com/app/service/ papkasini tekshirib, " \\
  "@Transactional annotatsiyasi yetishayotgan metodlarni sanab ber" \\
  --output-format json

# Misol: TypeScript type xatolarini topish
claude -p "@src/types/api.ts ni o'qib, 'any' tipi ishlatilgan" \\
  "barcha joylarni topib, to'g'ri tiplar bilan almashtir" \\
  --no-permission-prompts
```

JSON Chiqish Formati — Skriptda Qayta Ishlash

```
# Bash skriptda JSON natijani qayta ishlash:
RESULT=$(claude -p "pom.xml ni tekshirib, eskirgan dependency'larni sanab
ber" \\
  --output-format json)

# jq bilan parse qilish:
echo $RESULT | jq '.result'

# Node.js skriptda:
const { execSync } = require('child_process');
const result = JSON.parse(
  execSync('claude -p "src/ ni tekshir" --output-format json')
);
console.log(result.result);
```

Stdin orqali ma'lumot uzatish

```
# Fayl mazmunini stdin orqali yuborish:
cat src/main/java/com/app/UserService.java | \\
  claude -p "Bu kodni ko'rib, SOLID prinsiplarini buzgan joylarni ko'rsat"

# Git diff ni tekshirish:
git diff HEAD~1 | claude -p "Bu o'zgarishlarni code review qilib ber"
```

10.2 GitHub Actions Integratsiyasi

Java Maven + Claude Code CI Pipeline

```
# .github/workflows/claude-review.yml
name: Claude Code Review

on:
  pull_request:
    branches: [main, develop]
    paths:
      - 'src/**/*.java'

jobs:
  code-review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Install Claude Code
        run: npm install -g @anthropic-ai/claude-code

      - name: Get changed Java files
        id: changed
        run: |
          echo 'files<<EOF' >> $GITHUB_OUTPUT
          git diff --name-only origin/main...HEAD -- '*.java' >>
$GITHUB_OUTPUT
          echo 'EOF' >> $GITHUB_OUTPUT

      - name: Claude Code Review
        env:
          ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
        run: |
          git diff origin/main...HEAD -- '*.java' | \
          claude -p "Bu Java PR o'zgarishlarini code review qilib ber.
          NullPointerException xavfi, @Transactional yetishmasligi,
```

```
          N+1 query va security muammolarini topib, markdown jadvalda
ko'rsat."
          --output-format text > review.md

      - name: Post Review Comment
        uses: actions/github-script@v7
        with:
          script: |
            const fs = require('fs');
            const review = fs.readFileSync('review.md', 'utf8');
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## Claude Code Review\n\n' + review
            });
```

TypeScript + Node.js CI Pipeline

```
# .github/workflows/ts-claude-check.yml
name: TypeScript Claude Check

on:
  pull_request:
    paths: ['src/**/*.ts', 'src/**/*.js']

jobs:
  quality-check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - run: npm ci
      - run: npm install -g @anthropic-ai/claude-code

      - name: Type Check + Claude Analysis
        env:
          ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
        run: |
          # Avval TypeScript compile tekshiruvi
          npx tsc --noEmit

          # Keyin Claude bilan chuqur tahlil
          git diff origin/main...HEAD -- '*.ts' '*.js' | \
          claude -p "Bu TS/JS o'zgarishlarni tekshir:
          1) 'any' tipi ishlatilganmi?
          2) Unhandled promise rejection bormi?
          3) Security muammolar (SQL injection, XSS)?
          4) Test coverage yetarliimi?
          Muammolar bo'lsa markdown ro'yxatda ko'rsat."
          --output-format text
```

10.3 Docker ichida Claude Code

```
# Dockerfile -- Claude Code bilan
FROM node:20-slim

# Claude Code o'rnatish
RUN npm install -g @anthropic-ai/claude-code

# Java uchun qo'shimcha:
# FROM eclipse-temurin:21-jdk-slim
# RUN apt-get update && apt-get install -y nodejs npm
# RUN npm install -g @anthropic-ai/claude-code

WORKDIR /app
COPY . .

# Headless rejimda ishlatish
CMD ["claude", "-p", "src/ ni tekshirib, muammolarni ko'rsat", \
     "--output-format", "json", "--no-permission-prompts"]
```

```
# Docker Compose bilan CI/CD:
# docker-compose.yml
services:
  claude-review:
    build: .
    environment:
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
    volumes:
      - ./src:/app/src:ro
    command: >
      claude -p "src/ papkasini to'liq o'qib,
      kodni baholab, xulosa hisobot ber"
      --output-format json --no-permission-prompts
```

11. ENTERPRISE VA JAMOALAR UCHUN SOZLAMA

11.1 Settings Ierarxiyasi va managed-settings.json

```
# Ustuvorlik tartibi (yuqoridan pastga):
1. Buyruq qatori flaglari              # --model opus
2. .claude/settings.local.json         # Shaxsiy, git'ga kirmaydi
3. .claude/settings.json               # Jamoa, git'ga kiradi
4. ~/.claude/settings.local.json       # Global shaxsiy override
5. ~/.claude/settings.json             # Global shaxsiy default
   managed-settings.json              # Tashkilot siyosati (eng kuchli)
```

managed-settings.json — Tashkilot Siyosati

```
// Tashkilot adminlari tomonidan boshqariladi
// Foydalanuvchilar bu sozlamalarni o'zgartira olmaydi
{
  "permissions": {
    "allow": [
      "Bash(git *)",
      "Bash(mvn *)",
      "Bash(npm *)",
      "Edit(src/**)",
      "Read(**/*)",
    ],
    "deny": [
      "Bash(rm -rf *)",
      "Bash(curl * | bash)",
      "Read(.env)",
      "Read(**/*.key)",
      "Read(**/*.pem)"
    ]
  },
  "disableTools": ["WebSearch"],
  "defaultModel": "sonnet"
}
```

11.2 Jamoalar Uchun settings.json

```
// .claude/settings.json (git'ga kiradi -- jamoa ulashadi)
{
  "agent": "java-backend",
  "spinnerVerbs": {
    "mode": "replace",
    "verbs": ["Tahlil qilmoqda", "Yaratmoqda", "Tekshirmoqda",
"Optimallashtirmoqda"]
  },
  "statusLine": {
```

```
    "type": "command",
    "command": "echo $(git branch --show-current) [$(git log -1
--format='%s' | cut -c1-30)]"
  },
  "hooks": {
    "PostToolUse": [{
      "matcher": {"tool_name": "Write", "file_pattern": "**/*.java"},
      "hooks": [{"type": "command", "command": "mvn checkstyle:check -q"}]
    },{
      "matcher": {"tool_name": "Write", "file_pattern": "**/*.ts"},
      "hooks": [{"type": "command", "command": "npx prettier --write
$FILE"}]
    }]
  }
}
```

11.3 Parallel Agentlar — Ko'p Vazifalarni Bir Vaqtda

Claude bir nechta subagentni parallel ishga tushira oladi — bu katta loyihalarda vaqtni sezilarli kamaytiradi:

```
# .claude/commands/full-feature.md
---
description: Java backend + TS frontend + testlar parallel yaratish
model: opus
---

# Parallel Feature Yaratish

Quyidagi vazifalarni PARALLEL bajar:

Task 1 (java-backend agent):
  - Entity, Repository, Service, Controller yaratish
  - JUnit testlari

Task 2 (ts-frontend agent):
  - React komponent va hook yaratish
  - API service integration

Task 3 (js-node agent -- agar API gateway bo'lsa):
  - Express route va middleware
  - Jest integration testlari

Barcha parallel tasklar tugagach:
- code-reviewer agenti bilan natijalarni ko'rib chiq
- Integratsiya testlarini ishga tushir
- Xulosa hisobot ber
```

MASLAHAT

Parallel agentlar har biri izolyatsiyalangan kontekstda ishlaydi. Ular bir-birining o'zgarishlarini ko'rmaydi. Integratsiya asosiy agent tomonidan amalga oshiriladi.

11.4 Plugin Marketplace va LSP Jamoaviy Sozlama

```
// .claude/settings.json -- jamoa uchun LSP plugin'larni avtomatik
o'rnatish
{
  "enabledPlugins": [
    "jdtls@claude-code-lsps",
    "vtsls@claude-code-lsps"
  ],
  "extraKnownMarketplaces": [
    {
      "name": "company-internal",
      "url": "https://plugins.company.internal/claude"
    }
  ]
}

# Yangi jamoa a'zosi loyihani clone qilib 'claude' ni ishga tushirganda,
# Claude Code avtomatik ravishda plugin'larni o'rnatishni taklif etadi.
```

12. HAQIQIY ISH SENARIYLARI

12.1 Bug Fix Oqimi — Java NullPointerException

```
# 1. Plan rejimida tushunish
> /plan
> UserService.java da NullPointerException kelib chiqmoqda.
  Stack trace:
  java.lang.NullPointerException: Cannot invoke
  'com.app.model.User.getEmail()' because 'user' is null
  at UserService.sendWelcomeEmail(UserService.java:84)

# Claude bajaradi:
# - UserService.java 84-qatorni o'rganadi
# - UserRepository.findById() qaytaruvchi tipini tekshiradi
# - Optional<User> ishlatilmayotganini aniqlaydi

# 2. Tasdiqlangan tuzatish
> UserRepository.findById() Optional<User> qaytaradi,
  lekin UserService.java 84-qatorda orElseThrow ishlatilmagan.
  userNotFoundException bilan orElseThrow qo'sh,
  GlobalExceptionHandler.java dagi pattern'ga amal qil.

# 3. Test
> testSendWelcomeEmail_UserNotFound_ThrowsException() metodini yoz

# 4. Commit
> fix(service): handle null user in sendWelcomeEmail xabari bilan commit
qil
```

12.2 Code Review Oqimi — TypeScript PR

```
# Avtomatik code review (CI/CD'da yoki qo'lda):

# Qo'lda -- code-reviewer agentini ishga tushirish:
> /plan
> code-reviewer agenti bilan src/controllers/orderController.ts ni
  va src/services/orderService.ts ni ko'rib chiq.
  Async/await xatolari, type safety, security muammolarini topib ber.

# Claude code-reviewer agentini subagent sifatida chaqiradi:
# Task(subagent_type='code-reviewer',
#      prompt='orderController.ts va orderService.ts ni review qil')

# CI/CD'da avtomatik (GitHub Actions bilan):
git diff main...feature-branch -- '*.ts' | \\
claude -p "Bu TypeScript o'zgarishlarni ko'rib chiq:
  1) 'any' tipi bormi? 2) Promise rejection bor? 3) Zod validatsiya yo'qmi?
  Markdown jadvalda ko'rsat" --output-format text
```

12.3 Yangi Feature Oqimi — JavaScript Node.js

```
# 1. Plan rejimida arxitekturani muhokama qilish
> /plan
> routes/orderRoutes.js va controllers/orderController.js ni ko'rib,
  yangi 'discount' feature uchun nima kerakligini reja tuzib ber.
  Mavjud product discount pattern'iga amal qil.

# Claude taklif etadi:
# - models/Discount.js (Mongoose model)
# - services/discountService.js
# - controllers/discountController.js
# - routes/discountRoutes.js
# - __tests__/discountController.test.js

# 2. Tasdiqlash va bajarish
> Tasdiqlading. Bosqichma-bosqich: avval model, keyin service.

# 3. Har bosqichdan keyin test va commit
> Jest testlar o'tganidan keyin commit qil: feat(discount): add Discount
model

# 4. /compact agar context to'lib qolsa
> /compact
```

12.4 Katta Refactoring — Java Spring Boot

```
# Katta refactoring uchun parallel agentlar strategiyasi:

# 1. Avval to'liq tushunish
> /plan
> src/main/java/com/app/ papkasini skanlab,
  service qatlamida mavjud anti-pattern'larni ko'rsat.
  (N+1 query, God class, missing @Transactional)

# 2. Parallel refactoring (alohida branchlarda)
> /add-feature buyrug'ini ishga tushir
  (backend agent N+1 query, frontend agent correspondingly)

# 3. Har submoduldan keyin test
> mvn test -pl :user-module buyrug'ini ishga tushir va
  xatolarni tuzat

# 4. Integratsiya
> mvn test buyrug'ini ishga tushirib to'liq test o'tkazib ber
```

13. BARCHA SLASH BUYRUQLAR — TO'LIQ MA'LUMOTNOMA

13.1 Sessiya Boshqaruvi

Buyruq	Tavsif
/clear	Suhbat tarixini to'liq tozalash. Yangi mavzuga o'tganda.
/compact	Suhbat tarixini siqib xulosa saqlaydi. Context ~50% da qo'lda ishlating.
/resume	So'nggi sessiyani davom ettirish. 'claude --resume' CLI'dan ham ishlaydi.
/rewind	Suhbat va/yoki kodga oldingi nuqtaga qaytish. Noto'g'ri burilishda.

/clear = yangi mavzu. **/rewind** = xato qaror. **/compact** = bir mavzu, context tugaydi.

13.2 Model va Rejim

Buyruq	Tavsif
/model	Model pickerni ochish. Opus 4.6, Sonnet 4.5, Haiku 4.5 orasidan tanlash.
/fast	Tezkor rejim. Bir xil Opus 4.6, tezroq token chiqishi. Model o'zgarmaydi.
/plan	Plan rejimi. Claude faqat o'qiydi, reja taklif etadi. KOD YOZMAYDI.
/config	Interaktiv sozlamalar: mavzu, output uslubi, bildirishnoma, ruxsatlar.

13.3 Loyiha Boshqaruvi

Buyruq	Tavsif
/init	CLAUDE.md yaratish. Loyiha tuzilmasini skanlab boshlang'ich fayl hosil qiladi.
/memory	CLAUDE.md xotira fayllarini redaktorda ochish. O'zgarishlar saqlanadi.
/permissions	Claude nima qila olishini boshqarish. Wildcard sintaksisni qo'llab-quvvatlaydi.
/sandbox	Bash buyruqlar uchun fayl/tarmoq izolyatsiyasi. Xavfsizroq, kamroq ruxsat.
/agents	Agentlarni ko'rish, yaratish, tahrirlash va o'chirish.

Buyruq	Tavsif
/mcp	MCP serverlarni ko'rish, qo'shish, olib tashlash va tekshirish.
/plugin	Plugin marketplace: o'rnatish, o'chirish, yangilash, kashf etish.

13.4 Ma'lumot va Nosozlik Tuzatish

Buyruq	Tavsif
/context	Context oynasini rangli grid ko'rsatish — nima qancha joy olgan.
/cost	Joriy sessiya uchun token sarfi va xarajat ko'rsatish.
/doctor	O'rnatmani to'liq tekshirish: auth, sozlama, ruxsatlar, yangilash.
/help	Barcha slash buyruqlarni ko'rsatish.

13.5 Sozlash Buyruqlari

Buyruq	Tavsif
/keybindings	Klaviatura tugmalarini sozlash.
/terminal-setup	IDE terminallarida Shift+Enter ni yoqish.
/vim	Vim uslubida tahrirlash rejimini yoqish.
/statusline	Kompozitor ostida ma'lumot satrini sozlash (model, context, git branch).

Permissions Misollar

```
// Ruxsat berish (Java/JS/TS loyiha uchun):
Bash(git *)            // Barcha git buyruqlari
Bash(mvn *)            // Barcha Maven buyruqlari
Bash(npm *)            // Barcha npm buyruqlari
Bash(npx tsc *)        // TypeScript compile
Bash(npx jest *)       // Jest testlar
Edit(src/**)           // src/ dagi har qanday faylni tahrirlash

// Bloklash:
Bash(rm -rf *)         // Halokatli buyruqlarni bloklash
Read(.env)             // Maxfiy kalitlarni o'qishni bloklash
Read(**/*.key)         // SSH/API kalitlarni bloklash
```

14. IDE INTEGRATSIYA VA XARAJAT OPTIMALLASHTIRISH

14.1 VS Code ichidan Claude Code

Claude Code'ni VS Code'ning o'rnatilgan terminali orqali ishlatish eng qulay yondashuv:

```
# 1. VS Code terminalni ochish:
Ctrl+` (backtick)  yoki  View -> Terminal

# 2. Claude Code'ni ishga tushirish:
cd ./mening-loyiham && claude

# 3. Shift+Enter ni yoqish (IDE terminali uchun):
> /terminal-setup
```

VS Code + Claude Code Optimal Sozlama

```
// .vscode/settings.json
{
  "terminal.integrated.fontSize": 13,
  "terminal.integrated.env.osx": {
    "ANTHROPIC_API_KEY": "${env:ANTHROPIC_API_KEY}"
  },
  // Split terminal: bir tomonda Claude Code, bir tomonda dev server
  "terminal.integrated.splitCwd": "workspaceRoot"
}
```

VS Code Workflow — TypeScript Misoli

```
# Terminal 1: Claude Code (ts-frontend agenti bilan)
claude --agent ts-frontend

# Terminal 2: Dev server (parallel ishlaydi)
npm run dev

# VS Code editor'da @src/controllers/userController.ts ochiq
# Claude terminali'da:
> @src/controllers/userController.ts ni o'qib,
  'any' tiplarini to'g'ri TypeScript tiplari bilan almashtir.
  Mavjud ApiResponse<T> generic tipidan foydalansin.

# Claude o'zgartirish kiritadi -> VS Code avtomatik yangilanadi
# Terminal 2: Hot reload, o'zgarishlar darhol ko'rinadi
```

14.2 IntelliJ IDEA ichidan Claude Code

```
# 1. IntelliJ terminalni ochish:
Alt+F12 (Windows/Linux)  yoki  Option+F12 (macOS)

# 2. Claude Code ishga tushirish:
claude --agent java-backend

# 3. Shift+Enter yoqish:
> /terminal-setup
```

IntelliJ + Claude Code Workflow — Java Misoli

```
# IntelliJ terminali'da:
> /plan
> @src/main/java/com/app/service/OrderService.java ni ko'rib,
  calculateDiscount() ga yangi 'BULK' discount turi qo'sh.
  DiscountType enum'ga qiymat qo'sh,
  OrderServiceTest.java dagi pattern'ga mos test yoz.

# Claude reja taklif etadi -> tasdiqlaysiz -> bajaradi
# IntelliJ editor'da: o'zgarishlar darhol ko'rinadi
# IntelliJ Git: diff'ni ko'rib commit qilasiz
```

Muammo	Yechim
Shift+Enter ishlamayapti	/terminal-setup buyrug'ini ishga tushiring
ANTHROPIC_API_KEY topilmayapti	~/.zshrc yoki ~/.bashrc ga export qo'shing, terminalni qayta oching
Claude va siz bir faylda konflikti	Claude ishlayotganda shu faylni tahrirlashdan saqlaning
IntelliJ indeksatsiya sekinlashdi	Claude yozgan fayllar avtomatik indekslanadi — bu normaldir

14.3 Xarajat Optimallashtirish Strategiyasi

To'g'ri model tanlash bilan bir xil natijani 3-5 baravar arzonroq olish mumkin.

Vazifa Turi	Tavsiya Etilgan Model	Sabab
Murakkab arxitektura (Spring Security, JWT, migration)	Opus 4.6 High	Chuqur fikrlash kerak

Vazifa Turi	Tavsiya Etilgan Model	Sabab
Yangi feature (endpoint, komponent)	Opus 4.6 Medium yoki Sonnet 4.5	Balans: sifat/narx
Kunlik bug fix, maydon qo'shish	Sonnet 4.5	Tez, sifatli, arzon
Fayl o'qish, tushuntirish, qidirish	Haiku 4.5	Oddiy vazifa — tejamkor
Test yozish (pattern mavjud)	Haiku 4.5 yoki Sonnet 4.5	Pattern'dan nusxa — murakkab emas
CI/CD PR review (headless, tez-tez)	Sonnet 4.5	Avtomatik, ko'p chaqiriladi — Opus qimmat

Token Tejash Texnikalari

- **/compact qo'lda ~50% da:** Avtomatik compact muhim kontekstni yo'qotishi mumkin. Qo'lda boshqarsangiz nazorat sizda
- **Aniq @fayl referenslar:** '@src/service/UserService.java' — butun src/ ni o'qitishdan 10x arzonroq
- **Kichik, aniq vazifalar:** To'g'ri birinchi urinishda bajarish ortiqcha retry'larni kamaytiradi
- **Haiku'dan maksimal foydalaning:** Fayl o'qish, qidirish, tushuntirish — barchasida Haiku yetarli
- **CI/CD'da Sonnet:** GitHub Actions'da har PR uchun Opus qimmat. Sonnet yetarli darajada sifatli

Ssenariy	Asosiy Model	Oylik Xarajat (taxminiy)
Individual dasturchi, kunlik foydalanish	Sonnet 4.5 asosan	~$15-25/oy
Java katta refactoring loyiha	Opus 4.6 + Sonnet aralash	~$50-80/oy
CI/CD PR review (20 PR/kun)	Sonnet 4.5 headless	~$20-30/oy
Pro obuna ($20/oy flat)	Haftalik limit ichida barchasi	$20/oy (samarali)
Max 5x obuna ($100/oy flat)	Cheksiz (haftalik limit 5x)	$100/oy (og'ir foydalanish)

MUHIM

Haftasiga 10+ soat ishlatsangiz API Key o'rniga Max 5x obuna ($100/oy) tejamkorroq bo'lishi mumkin. /cost buyrug'i bilan kuzatib boring.

```
● ● ●      claude — /cost

> /cost

  Session Cost Summary
  ──────────────────────────────────────
  Input tokens:    45,230   →  $0.14
  Output tokens:    8,120   →  $0.12
  Cache read:      12,500   →  $0.01
  ──────────────────────────────────────
  Total estimated:          $0.27

  Model: claude-sonnet-4-5
  Duration: 18 min 42 sec
```

↑ */cost — joriy sessiya token sarfi va taxminiy xarajat*

```
● ● ●      claude — /context

> /context

  Context Window Usage  [└└└
```

15. OLTIN QOIDALAR VA CHEAT SHEET

15.1 Professional Foydalanishning 8 Oltin Qoidasi

1. DOIM /PLAN BILAN BOSHLANG

2-3 fayldan ortiq yoki arxitektura qarorlarini talab qiladigan BARCHA vazifalar uchun. Spring Security, JWT, migration — barchasida /plan.

2. CLAUDE.MD NI 150 QATORDAN QISQA SAQLANG

Ko'proq joy kerak bo'lsa — Rules (.claude/rules/) va Skills (.claude/skills/) ishlating. Uzun CLAUDE.md ko'rsatmalarni suyultiradi.

3. /COMPACT NI ~50% DA QO'LDA ISHLATNG

Avtomatik compact'ni kutmang. mvn build output, katta Java fayllari tez to'ldiradi. Nazorat sizda qolsin.

4. HAR KICHIK VAZIFADAN KEYIN COMMIT QILING

feat(auth): add JWT, fix(service): handle null user — kichik, Conventional Commits uslubida. Bitta mega-commit xavfli.

5. BIR VAQTDA BITTA VAZIFA

Kichik, aniq so'rovlar mega-promptlardan ko'ra yaxshi natija beradi. 'UserService ga 5 ta o'zgartirish qil' o'rniga bitta bitta.

6. FAYLLARNI ANIQ BELGILANG

'Xatoni to'g'irla' emas -- 'src/main/java/com/app/service/UserService.java 84-qatordagi null xatoni tuzat' deb yozing.

7. PATTERN'LARGA DOIM HAVOLA QILING

Java: 'ProductService.java dagi uslubda'. TypeScript: 'productController.ts pattern'iga amal qil'. Bu boshidanoq to'g'ri kod yozdirishning kalitidir.

8. CI/CD GA INTEGRATSIYA QILING

claude -p "git diff | review qil" --output-format json bilan PR'larni avtomatik tekshiring. Jamoangizning sifat standarti oshadi.

15.2 Barcha Buyruqlar Jadvali

Kategoriya	Buyruqlar
Sessiya	/clear /compact /resume /rewind
Model & Rejim	/model /fast /plan /config
Loyiha	/init /memory /permissions /sandbox /agents /mcp /plugin
Ma'lumot	/context /cost /doctor /help
Sozlash	/keybindings /terminal-setup /vim /statusline

15.3 Klaviatura Tugmalari

Tugma	Amal
Enter	Prompt yuborish
Shift+Enter yoki \\	Yangi qator
Ctrl+C	Bekor qilish
Ctrl+D / /exit	Chiqish
Option+P / Alt+P	Model tanlash
Ctrl+V	Screenshot joylashtirish
Esc Esc	Rewind
Ctrl+G	Redaktorda ochish

Tugma	Amal
Shift+Tab	Ruxsatlarni almashtirish
!buyruq	Claude tahlilisiz bash bajarish

15.4 Keyingi Qadamlar

1. **O'rnatish:** npm install -g @anthropic-ai/claude-code && claude
2. **/init:** Loyihangiz uchun CLAUDE.md yarating (Java/TS/JS konventsiyalar)
3. **/plan bilan birinchi real vazifani bajaring** — kichik bug yoki yangi endpoint
4. **Birinchi skill yarating** — java-spring-patterns yoki ts-api-patterns
5. **Til-specific agent yarating** — java-backend, ts-frontend, js-node
6. **MCP va LSP ulang** — GitHub, PostgreSQL, jdtls/vtsls
7. **CI/CD integratsiya qiling** — GitHub Actions'da claude -p bilan PR review
8. **IDE sozlang** — VS Code yoki IntelliJ da /terminal-setup, output style'ni belgilang

15.5 Rasmiy Manbalar

Manba	URL
Rasmiy Hujjat	https://docs.claude.com/en/docs/claude-code/overview
Claude Code Docs	https://code.claude.com/docs/en
npm Package	https://www.npmjs.com/package/@anthropic-ai/claude-code
LSP Plugins	github.com/Piebald-AI/claude-code-lsps
cclsp (MCP+LSP)	github.com/ktnyt/cclsp

XULOSA

Claude Code bu faqat AI chatbot emas -- u sizning dasturiy ta'minot muhandisingiz.

```
CLAUDE.md      -> loyiha konteksti (Java/TS/JS konventsiyalar)
Rules          -> yo'l-scoped ko'rsatmalar
Skills         -> java-spring-patterns, ts-api-patterns bilimlari
Agents         -> java-backend, ts-frontend, js-node maxsus agentlar
Commands       -> /add-feature, /code-review murakkab workflow'lar
Hooks          -> Checkstyle, Prettier, ESLint avtomatik
```

```
MCP + LSP      -> GitHub, PostgreSQL, jdtls, vtsls integratsiya
CI/CD          -> GitHub Actions bilan PR avtomatik review
IDE            -> VS Code, IntelliJ IDEA bilan integratsiya
Voice Hooks    -> Ovozli boshqarish, qollar band uchun
Output Styles  -> Explanatory, Learning, Custom javob uslubi
Xarajat        -> To'g'ri model tanlash = 3-5x tejash
Tool tanlash   -> Claude Code vs Cursor vs Antigravity real benchmark

0% Vibe Coding dan 100% Professional Agentic Engineering ga xush kelibsiz.
```

16. REAL LOYIHA: JAVA SPRING + POSTGRESQL + VUE 3 + TYPESCRIPT

SAVIYA: 0% -> 100% Noldan to'liq full-stack loyihani Claude Code bilan qurish

LOYIHA: TaskManager Full-Stack

Backend: Java 21 + Spring Boot 3.2 + Spring Security + PostgreSQL
Frontend: Vue 3 + TypeScript + Vite + Pinia + Tailwind CSS
Auth: JWT token autentifikatsiyasi
API: REST, Swagger/OpenAPI dokumentatsiya
Tests: JUnit 5 + Vitest
Deploy: Docker Compose

QADAM 1: Loyiha Tuzilmasini Yaratish

Spring Initializr va Vite bilan asosiy tuzilma yaratiladi:

```
# 1. Backend — Spring Boot loyihasini yaratish
# https://start.spring.io/ dan yoki curl bilan:
curl https://start.spring.io/starter.zip \
  -d dependencies=web,data-jpa,security,postgresql,validation,lombok \
  -d javaVersion=21 -d bootVersion=3.2.0 \
  -d groupId=com.taskmanager -d artifactId=backend \
  -o backend.zip && unzip backend.zip -d backend

# 2. Frontend — Vue 3 + TypeScript loyihasini yaratish
npm create vue@latest frontend
# Tanlash: TypeScript: Yes | Vue Router: Yes | Pinia: Yes
# Vitest: Yes | ESLint: Yes | Prettier: Yes
cd frontend && npm install
npm install -D tailwindcss autoprefixer && npx tailwindcss init
```

terminal — loyiha tuzilmasi

```
taskmanager/
  backend/
    src/main/java/com/taskmanager/
      controller/    # REST endpoint'lar
      service/       # Biznes mantiq
      repository/    # JPA repository'lar
      model/         # JPA entity'lar
      dto/           # Request/Response DTO'lar
      config/        # Security, Swagger konfiguratsiya
      exception/     # Global exception handling
```

```
    src/main/resources/
      application.yml
    pom.xml
  frontend/
    src/
      components/    # Vue komponentlar
      views/         # Sahifalar
      stores/        # Pinia store'lar
      services/      # API chaqiruvlar
      types/         # TypeScript interfeys'lar
    vite.config.ts
  .claude/           # Claude Code sozlamalari
  docker-compose.yml
```

↑ *Loyiha tuzilmasi — backend va frontend alohida papkalarda*

QADAM 2: Claude Code ni Ishga Tushirish va /init

```
cd taskmanager
claude

> /init
# Claude loyiha tuzilmasini skanlab CLAUDE.md yaratadi
```

```
● ● ●      claude — /init natijasi

> /init

  Scanning project structure...
  Found: backend/ (Java 21, Spring Boot 3.2)
  Found: frontend/ (Vue 3, TypeScript, Vite)
  Found: docker-compose.yml

  Generated CLAUDE.md with:
  ✓ Project overview
  ✓ Build commands
  ✓ Directory structure
  ✓ Technology stack

  CLAUDE.md created (42 lines)
  Edit it to add your conventions and patterns.
```

↑ */init — loyiha tuzilmasini avtomatik skanlab CLAUDE.md yaratadi*

Claude yaratgan CLAUDE.md ni o'zingiz **konventsiyalar va pattern'lar** bilan to'ldirasiz:

```
# CLAUDE.md (to'ldirilgan holat)

## Loyiha: TaskManager Full-Stack
Java 21 + Spring Boot 3.2 + PostgreSQL | Vue 3 + TypeScript + Vite

## Buyruqlar
```

```
- cd backend && mvn spring-boot:run
- cd frontend && npm run dev
- docker-compose up -d   # PostgreSQL
- cd backend && mvn test
- cd frontend && npm run test:unit

## Konventsiyalar
- Backend: TaskService.java uslubiga amal qil
- Frontend: TaskList.vue ni namunaviy komponent sifatida kuzat
- Har endpoint uchun JUnit 5 testi majburiy
- Vue: Composition API, <script setup lang="ts"> sintaksis
- Pinia store uchun useTaskStore() pattern
```

QADAM 3: PostgreSQL va Spring konfiguratsiyasi

```
# docker-compose.yml — PostgreSQL
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: taskmanager
      POSTGRES_USER: taskuser
      POSTGRES_PASSWORD: secret
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
```

```
● ● ●      claude — database konfiguratsiyasi so'rovi

> /plan
> backend/src/main/resources/application.yml ni yaratib,
  Docker Compose PostgreSQL ga ulanishni sozla.
  JPA: create-drop (dev), show-sql: false.
  Swagger UI yoqilsin.

  Reading pom.xml...
  Reading src/main/resources/ ...

  REJA:
  1. application.yml — datasource, jpa, swagger sozlama
  2. SwaggerConfig.java — OpenAPI bean
  3. application-test.yml — H2 in-memory (testlar uchun)

  Tasdiqlaysizmi? (yes/no)
> yes

  Writing application.yml ... done
  Writing SwaggerConfig.java ... done
  Writing application-test.yml ... done
  ✓ 3 fayl yaratildi
```

↑ */plan — Claude avval reja taklif etadi, tasdiqlash so'raydi*

QADAM 4: Birinchi Entity va Repository — Task

```
● ● ●      claude — Task entity so'rovi

> @src/main/java/com/taskmanager/ tuzilmasini ko'rib,
  Task entity yaratib ber:
  - id (UUID), title, description, status (ENUM: TODO/IN_PROGRESS/DONE)
  - priority (LOW/MEDIUM/HIGH), dueDate, createdAt, updatedAt
  - User bilan ManyToOne munosabat
  Lombok @Builder, @Data, @Entity ishlatilsin.

  Writing model/Task.java ...
  Writing model/TaskStatus.java (enum) ...
  Writing model/TaskPriority.java (enum) ...
  Writing repository/TaskRepository.java ...

  ✓ Task.java (47 lines)
  ✓ TaskStatus.java
  ✓ TaskPriority.java
  ✓ TaskRepository.java — findByUser, findByStatus methods
```

↑ *Entity va Repository bir so'rov bilan yaratiladi*

```
# Yaratilgan Task.java (Claude tomonidan):
@Entity @Table(name = "tasks")
@Data @Builder @NoArgsConstructor @AllArgsConstructor
public class Task {
    @Id @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @NotBlank @Size(max = 200)
    private String title;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private TaskStatus status = TaskStatus.TODO;

    @Enumerated(EnumType.STRING)
    private TaskPriority priority = TaskPriority.MEDIUM;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @CreationTimestamp
    private LocalDateTime createdAt;
}
```

QADAM 5: Service va Controller

```
● ● ●      claude — Service va Controller

> @src/main/java/com/taskmanager/model/Task.java va
  @src/main/java/com/taskmanager/repository/TaskRepository.java ni
  ko'rib, TaskService va TaskController yaratib ber.
  CRUD operatsiyalari: getAll, getById, create, update, delete.
  Swagger @Operation annotatsiyalar qo'shilsin.
  GlobalExceptionHandler.java ham kerak.

  Writing service/TaskService.java ...
  Writing controller/TaskController.java ...
  Writing dto/TaskCreateRequest.java ...
  Writing dto/TaskResponse.java ...
  Writing exception/GlobalExceptionHandler.java ...
  Writing exception/TaskNotFoundException.java ...

  ✓ 6 fayl yaratildi
  Running: mvn compile -q
  BUILD SUCCESS
```

↑ *Compile buyrug'i avtomatik ishga tushiriladi — xatolar darhol ko'rinadi*

```
● ● ●      claude — JUnit testlar

> TaskService uchun JUnit 5 testlar yoz.
  TaskServiceTest.java mavjud pattern'ga amal qilsin.
  @Mock TaskRepository, @InjectMocks TaskService.
  create, getById, update, delete uchun success va failure testlar.

  Writing test/TaskServiceTest.java ...

  Running: mvn test -pl . -Dtest=TaskServiceTest -q
  ........
  Tests run: 8, Failures: 0, Errors: 0, Skipped: 0
  BUILD SUCCESS

  ✓ 8 test muvaffaqiyatli o'tdi
```

↑ *Testlar yozilgandan so'ng darhol mvn test ishga tushiriladi*

QADAM 6: JWT Autentifikatsiya

```
> /plan
> Spring Security + JWT auth qo'sh.
  Register: POST /api/auth/register
  Login: POST /api/auth/login (JWT qaytarsin)
  /api/tasks/** endpoint'larini himoyala.
  jjwt library ishlatilsin (pom.xml da mavjud).
```

```
● ● ●      claude — JWT /plan natijasi

  Loyiha tekshirilmoqda...
  Reading pom.xml ... jjwt-api 0.12.3 topildi
```

```
  Reading config/ ... SecurityConfig.java yo'q
  Reading model/ ... User.java yo'q

  REJA (7 qadam):
  1. model/User.java  — UserDetails implementatsiyasi
  2. repository/UserRepository.java
  3. service/JwtService.java  — token generate/validate
  4. service/AuthService.java  — register/login
  5. controller/AuthController.java
  6. config/SecurityConfig.java  — filterChain
  7. filter/JwtAuthFilter.java

  Testlar ham yozilsinmi? (yes/no)
> yes

  ✓ 8 fayl yaratildi | Tests: 12/12 passed
  BUILD SUCCESS
```

↑ *JWT auth — 7 faylni to'g'ri tartibda yaratish rejasi*

QADAM 7: Vue 3 + TypeScript Frontend

```
cd taskmanager
claude --agent ts-frontend

> /plan
> Vue 3 + TypeScript frontend qur.
  Auth: login/register sahifalari (Pinia store).
  Tasks: ro'yxat, yaratish, tahrirlash, o'chirish.
  API: backend /api/auth va /api/tasks ga ulash.
  Tailwind CSS, responsive dizayn.
```

```
● ● ●      claude (ts-frontend agent) — Vue REJA

  Frontend tuzilmasi tekshirilmoqda...
  Reading vite.config.ts ...
  Reading src/ ... (bo'sh)

  REJA:
  1. types/index.ts — Task, User, ApiResponse<T> interfeys'lar
  2. services/api.ts — axios instance, interceptors (JWT header)
  3. services/authService.ts — login, register, logout
  4. services/taskService.ts — CRUD operatsiyalar
  5. stores/authStore.ts — Pinia, token saqlash
  6. stores/taskStore.ts — Pinia, task state
  7. views/LoginView.vue — login form
  8. views/RegisterView.vue
  9. views/TasksView.vue — asosiy sahifa
  10. components/TaskCard.vue
  11. components/TaskForm.vue
  12. router/index.ts — navigation guard

  Boshlaylikmi? (yes/no)
> yes
```

↑ *ts-frontend agenti Vue 3 loyiha uchun 12 fayllik reja taklif etadi*

```
● ● ●      claude — TypeScript types yaratish

  Writing src/types/index.ts ...

  // Yaratilgan typlar:
  export interface Task {
    id: string
    title: string
    status: 'TODO' | 'IN_PROGRESS' | 'DONE'
    priority: 'LOW' | 'MEDIUM' | 'HIGH'
    dueDate?: string
    createdAt: string
  }

  Writing src/services/api.ts ...
  Writing src/services/authService.ts ...
  Writing src/services/taskService.ts ...
  Writing src/stores/authStore.ts ...
  Writing src/stores/taskStore.ts ...

  Running: npx tsc --noEmit
  ✓ TypeScript compile — 0 xato
```

↑ *Strict TypeScript — barcha tiplar aniq, 'any' ishlatilmagan*

QADAM 8: TaskCard.vue Komponenti

```
● ● ●      claude — Vue komponent

> @src/types/index.ts ni ko'rib,
  TaskCard.vue komponenti yoz.
  Props: task: Task, emits: update, delete.
  Status badge (rang: TODO=kulrang, IN_PROGRESS=sariq, DONE=yashil).
  Priority icon. Tailwind CSS.
  <script setup lang="ts"> sintaksis.

  Writing src/components/TaskCard.vue ...

  Writing src/components/__tests__/TaskCard.test.ts ...
  Running: npm run test:unit -- TaskCard

  ✓ TaskCard.test.ts — 5/5 tests passed
```

↑ *Vue komponent + Vitest testi birgalikda yaratiladi*

```
<!-- Yaratilgan TaskCard.vue (qisqartirilgan) -->
<script setup lang="ts">
import type { Task } from '@/types'

const props = defineProps<{ task: Task }>()
const emit = defineEmits<{
  update: [task: Task]
```

```
  delete: [id: string]
}>()

const statusColors = {
  TODO: 'bg-gray-100 text-gray-700',
  IN_PROGRESS: 'bg-yellow-100 text-yellow-700',
  DONE: 'bg-green-100 text-green-700'
}
</script>

<template>
  <div class="bg-white rounded-lg shadow-sm p-4 border">
    <span :class="statusColors[task.status]">{{ task.status }}</span>
    <h3 class="font-medium mt-2">{{ task.title }}</h3>
    <div class="flex gap-2 mt-3">
      <button @click="emit('update', task)">Tahrirlash</button>
      <button @click="emit('delete', task.id)">O'chirish</button>
    </div>
  </div>
</template>
```

QADAM 8b: Vue Router — Navigation Guard

Authentication guard — foydalanuvchi login qilmagan bo'lsa **/login** sahifasiga yo'naltiradi:

```
// src/router/index.ts — to'liq kod
import { createRouter, createWebHistory } from 'vue-router'
import { useAuthStore } from '@/stores/authStore'

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      redirect: '/tasks'
    },
    {
      path: '/login',
      name: 'login',
      component: () => import('@/views/LoginView.vue'),
      meta: { requiresAuth: false }
    },
    {
      path: '/register',
      name: 'register',
      component: () => import('@/views/RegisterView.vue'),
      meta: { requiresAuth: false }
    },
    {
      path: '/tasks',
      name: 'tasks',
      component: () => import('@/views/TasksView.vue'),
      meta: { requiresAuth: true }
```

```
    },
    {
      path: '/:pathMatch(.*)*',
      redirect: '/tasks'
    }
  ]
})

// Navigation Guard — JWT token tekshiruvi
router.beforeEach((to, _from) => {
  const authStore = useAuthStore()

  // requiresAuth: true sahifalari uchun token tekshir
  if (to.meta.requiresAuth && !authStore.isAuthenticated) {
    return { name: 'login', query: { redirect: to.fullPath } }
  }

  // Allaqachon login qilgan foydalanuvchi /login ga kirsa — /tasks ga
  if (!to.meta.requiresAuth && authStore.isAuthenticated) {
    return { name: 'tasks' }
  }
})

export default router
```

```
● ● ●      claude — router/index.ts yaratilishi

> @src/stores/authStore.ts ni ko'rib,
  src/router/index.ts ni yaratib ber.
  Navigation guard: login qilmagan foydalanuvchi /tasks ga
  kirsa /login ga yo'naltirilsin.
  Login qilgan foydalanuvchi /login ga kirsa /tasks ga ketsin.
  Lazy loading barcha route'lar uchun.

  Reading src/stores/authStore.ts ...
    Found: isAuthenticated (computed), token (ref)

  Writing src/router/index.ts ...

  Running: npx tsc --noEmit
  ✓ TypeScript compile — 0 xato

  Navigation guard:
  GET /tasks  (token yo'q)  -> /login?redirect=/tasks
  GET /login  (token bor)   -> /tasks
  GET /tasks  (token bor)   -> /tasks  OK
```

↑ *Navigation guard — authStore.isAuthenticated computed property asosida yo'naltiradi*

QADAM 9: API Integratsiya va CORS Sozlama

```
● ● ●      claude — CORS xatosi tuzatish
```

```
# Frontend ishga tushirganda CORS xatosi:
# Access to XMLHttpRequest at 'http://localhost:8080/api/tasks'
# from origin 'http://localhost:5173' has been blocked by CORS

> Bu CORS xatosini tuzat.
  Backend: localhost:8080, Frontend: localhost:5173
  Faqat dev uchun emas, production uchun ham sozla.

  Reading config/SecurityConfig.java ...

  SecurityConfig.java ga corsConfigurationSource() qo'shilmoqda...
  application.yml ga cors.allowed-origins property qo'shilmoqda...

  ✓ CORS konfiguratsiyasi yangilandi
  Running: mvn compile -q ... BUILD SUCCESS
```

↑ *CORS xatosi — Claude error message'ni o'qib to'g'ri faylni topadi*

```
● ● ●      claude — Pinia + JWT interceptor

> @src/stores/authStore.ts va @src/services/api.ts ni ko'rib,
  JWT token axios interceptor'da har so'rovga qo'shilsinmi?
  Token muddati o'tsa logout qilsin.

  Reading src/stores/authStore.ts ...
  Reading src/services/api.ts ...

  api.ts ga request interceptor qo'shilmoqda:
    Authorization: Bearer {token}
  api.ts ga response interceptor qo'shilmoqda:
    401 -> authStore.logout() -> router.push('/login')

  Running: npx tsc --noEmit
  ✓ TypeScript compile — 0 xato
  npm run test:unit -- services
  ✓ 4/4 tests passed
```

↑ *JWT interceptor — barcha API chaqiruvlarga avtomatik token qo'shiladi*

QADAM 10: Docker Compose va Final Tekshiruv

```
● ● ●      claude — Docker Compose

> docker-compose.yml ga backend va frontend ham qo'sh.
  Backend Dockerfile yaratilsin.
  Frontend: nginx bilan production build.
  Hammasi bitta 'docker-compose up' bilan ishlasin.

  Writing backend/Dockerfile ...
  Writing frontend/Dockerfile ...
  Updating docker-compose.yml ...
  Writing nginx.conf ...

  ✓ 4 fayl yaratildi/yangilandi
```

↑ *Docker Compose — butun stack bitta buyruq bilan ishga tushadi*

```
# docker-compose.yml (to'liq):
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: taskmanager
      POSTGRES_USER: taskuser
      POSTGRES_PASSWORD: secret
    volumes:
      - postgres_data:/var/lib/postgresql/data

  backend:
    build: ./backend
    ports: ["8080:8080"]
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/taskmanager
      SPRING_DATASOURCE_USERNAME: taskuser
      SPRING_DATASOURCE_PASSWORD: secret
    depends_on: [postgres]

  frontend:
    build: ./frontend
    ports: ["80:80"]
    depends_on: [backend]

volumes:
  postgres_data:
```

```
● ● ●      claude — To'liq test natijasi

> Barcha testlarni ishga tushir va natijani xulosa qil.

  Running: cd backend && mvn test -q
  Tests run: 34, Failures: 0, Errors: 0, Skipped: 0
  BUILD SUCCESS

  Running: cd frontend && npm run test:unit
  ✓  TaskCard.test.ts (5)
  ✓  authService.test.ts (4)
  ✓  taskService.test.ts (6)
  ✓  taskStore.test.ts (5)
  Test Files  4 passed (4)
  Tests       20 passed (20)

  XULOSA:
  Backend: 34 JUnit test — PASSED
  Frontend: 20 Vitest test — PASSED
  Docker: barcha servislar ishlamoqda
  ✓ TaskManager loyihasi production uchun tayyor
```

↑ *To'liq test natijasi — backend 34 + frontend 20 = 54 test, hammasi o'tdi*

MUHIM

NATIJA: 10 qadamda noldan to'liq full-stack loyiha: 34 JUnit test + 20 Vitest test. Claude Code bilan bir ish kunida!

Best Practices — Loyiha Davomida Nima Qildik

Qadam	Texnika	Natija
1-2	/init + CLAUDE.md to'ldirish	Claude loyiha pattern'larini tanidi
3-4	/plan + tasdiqlash	Noto'g'ri arxitektura oldini olindi
5-6	Har fayldan keyin mvn compile	Xatolar darhol topildi, katta muammo bo'lmadi
6	JWT uchun /plan (7 fayl)	To'g'ri tartibda, bog'liqliklar hisobga olindi
7	--agent ts-frontend	Vue konventsiyalari avtomatik, Pinia pattern to'g'ri
8-9	@fayl referenslar	Claude kerakli faylni o'qib, moslikni ta'minladi
9	Error message to'liq berish	CORS xatosi 1 so'rovda tuzatildi
10	Barcha testlar yakunda	Integrasiya muammolari topildi va tuzatildi

Vaqt Taqqoslash

Bosqich	Qo'lda vs Claude Code bilan
Loyiha tuzilmasi	Qo'lda: 1-2 soat → Claude: 15 daqiqa
Task CRUD (back+front)	Qo'lda: 2-3 soat → Claude: 45 daqiqa
JWT autentifikatsiya	Qo'lda: 3-4 soat → Claude: 30 daqiqa
54 ta test	Qo'lda: 3-4 soat → Claude: 20 daqiqa
Docker konfiguratsiya	Qo'lda: 1-2 soat → Claude: 10 daqiqa
JAMI	Qo'lda: ~14 soat → Claude: ~2 soat (7x tezroq)

17. CLAUDE CODE vs CURSOR vs ANTIGRAVITY

Uchala tool ham **Claude modelini** ishlatishi mumkin. Lekin **natija tubdan farqli** — model emas, uni qanday ishlatish muhim.

ASOSIY SAVOL

Nima uchun bir xil Claude Opus, lekin Claude Code'da yaxshiroq ishlaydi? Cursor'da 188K token, Claude Code'da 33K token — bir xil vazifa, 5.5x farq. Sababi: Anthropic o'z agentini maxsus optimallashtirgan, Cursor esa universal wrapper ishlatadi.

17.1 Real Benchmark Raqamlari (2026 Fevral)

SWE-bench Verified — Haqiqiy GitHub Muammolarini Hal Qilish

SWE-bench haqiqiy open-source loyihalaridagi muammolarni hal qilishni o'lchaydi — **eng ishonchli benchmark.**

Tool	Model	SWE-bench	Token Sarfi	Narx/oy
Claude Code (Opus 4.5)	Claude Opus 4.5	80.9% ★	33K (1x)	$100-200
Claude Code (Sonnet 4.6)	Claude Sonnet 4.6	79.6%	~40K	$20
Gemini CLI	Gemini 3 Pro	80.6%	O'rta	Bepul
Antigravity	Gemini 3 Pro asosiy	76.2%	O'rta	Bepul (preview)
Cursor + Claude Opus	Claude Opus 4.5	~72%	188K (5.5x)	$20-200
Cursor (Composer)	O'z modeli	Tezroq, pastroq	Minimal	$20

DIQQAT

Cursor Claude Opus ishlatganda 188K token sarflaydi — Claude Code esa xuddi shu vazifani 33K token bilan bajaradi. Sifat ham past, xarajat ham yuqori.

17.2 Nima Uchun Farq Shuncha Katta?

Farq Sohasi	Claude Code	Cursor + Claude	Antigravity + Claude
Context	200K (haqiqiy, to'liq)	70-120K (qirqilgan, amaliy)	200K (Gemini) / qirqilgan (Claude)
Optimizatsiya	Anthropic maxsus tayyorlagan	Universal wrapper, ko'p model uchun	Google Gemini uchun tayyorlagan
Sistem prompt	Kodlash uchun maxsus	Universal, keng maqsadli	IDE uchun moslangan
Tool integratsiya	Bash, Git, fayl — chuqur	IDE orqali, vizual	IDE + brauzer + terminal
Token samaradorligi	5.5x tejamkor	Ko'p sarflaydi	O'rta

```
Cursor vs Claude Code — bir xil vazifa, farqli natija

# Bir xil vazifa: Next.js auth qo'shish

# Cursor (Claude Opus 4.5 bilan):
  Token sarfi: 188,234
  Xatolar: 3 (manual tuzatish kerak)
  Vaqt: ~12 daqiqa
  Narx: ~$0.85

# Claude Code (Claude Opus 4.5 bilan):
  Token sarfi: 33,102
  Xatolar: 0
  Vaqt: ~8 daqiqa
  Narx: ~$0.15

# Xulosa: 5.5x tejamkor, 0 xato, tezroq
```

↑ *Bir xil Opus modeli — lekin natija tubdan farqli*

17.3 Uchala Toolni Chuqur Taqqoslash

Claude Code

Jihati	Batafsil
Arxitektura	Terminal-first CLI agent — bash, git, fayl tizimi bilan to'g'ridan-to'g'ri ishlaydi
Model	Faqat Claude (Opus 4.6, Sonnet 4.6, Haiku 4.5) — boshqa model yo'q

Jihati	Batafsil
Context	200K haqiqiy (1M beta Opus 4.6), hech qanday qirqish yo'q
Alohida afzalligi	Code Security skaneri, MCP integratsiya, Agent Teams, CI/CD pipe
Kamchiligi	Tab autocomplete yo'q, terminal bilishi kerak, narxi yuqori
Narx	$20/oy (Sonnet) | $100/oy (5x) | $200/oy (20x) | API key (token hisobi)
Eng yaxshi holat	Murakkab refactoring, bug debugging, katta codebase, CI/CD

Cursor

Jihati	Batafsil
Arxitektura	AI-native IDE (VS Code fork) — inline AI, chat panel, background agents
Model	Claude Opus/Sonnet, GPT-5, Gemini, o'z Composer modeli (4x tezroq)
Context	Reklama: 200K | Amaliy: 70-120K (ichki qirqish bor)
Alohida afzalligi	Tab autocomplete, tezkor Composer modeli, Shadow Workspace, parallel 8 agent
Kamchiligi	Context qirqilgan, kredit tizimi murakkab, og'ir foydalanishda $60-80/oy
Narx	$0 (Free) | $20/oy (Pro) | $40/oy (Business) | $200/oy (Max)
Eng yaxshi holat	Kunlik feature, tez iteratsiya, yangi boshlovchi, tab autocomplete muhim

Google Antigravity

Jihati	Batafsil
Arxitektura	Agent-first IDE (VS Code/Windsurf fork) — Manager view + Editor view

Jihati	Batafsil
Model	Gemini 3 Pro (asosiy) + Claude Sonnet/Opus 4.5 + GPT-OSS 120B (tanlov)
Context	Gemini 3 Pro: 2M token (!) | Claude modeli: qirqilgan
Alohida afzalligi	Bepul (preview), parallel agent Manager view, brauzer agent, 2M context
Kamchiligi	Public preview — beqaror, jamoaviy funksiyalar 'coming soon', production uchun emas
Narx	Bepul (preview) | Kelajakda Google One AI Pro bilan integratsiya kutilmoqda
Eng yaxshi holat	Bepul sinab ko'rish, parallel agent workflow, katta JS/TS loyiha, tajriba

17.4 Nima Uchun Cursor Shuncha Mashhur?

Cursor SWE-bench'da Claude Code'dan past (72% vs 80.9%). Lekin 1 million+ foydalanuvchisi bor. Sabablari:

```
Cursor — tab autocomplete (Claude Code'da YO'Q)

# UserService.java ni yozayapsiz:
public User getUserBy|
#                     ^ kursor bu yerda

# Cursor darhol taklif beradi (50ms ichida):
  getUserById(UUID id) {
    return userRepository.findById(id)
      .orElseThrow(() -> new UserNotFoundException(id));
  }

# Tab bosasiz — tayyor!

# Claude Code'da bu funksiya YO'Q.
# Siz to'liq prompt yozasiz -> kutasiz -> natijani ko'rasiz.
```

↑ *Tab autocomplete — yozayotganda darhol taklif. Claude Code'da bu yo'q, shu sabab ko'pchilik Cursor'ni yaxshi ko'radi*

Sabab	Tushuntirish
Tab autocomplete	Yozayotganda darhol taklif — eng ko'p ishlatiladi, eng foydali

Sabab	Tushuntirish
Tanish interfeys	VS Code'dan o'tish oson — hech narsa o'rganish shart emas
Ko'p model tanlash	Claude, GPT-5, Gemini — bir joydan barcha modellar
Composer (o'z modeli)	4x tezroq oddiy vazifalar uchun — 30 sekundda tugaydi
Vizual diff	Har o'zgartishni ko'rib, qabul qilish yoki rad etish mumkin
Keng ekotizim	$29.3 milliard baholash, 1M+ foydalanuvchi, ko'p extension

MASLAHAT

Cursor + Claude Code birga ishlatish mumkin. Ko'pchilik kuchli dasturchilar: Cursor (kunlik) + Claude Code (og'ir muammolar). Bir-birini to'ldiradi.

17.5 Nima Uchun Ba'zilar Antigravity'ni Yaxshi Ko'radi?

```
Antigravity — Manager View (parallel 3 agent)

# Manager View'da 3 ta parallel agent:

  Agent 1 (Gemini 3 Pro):
  > Spring Boot backend — Entity, Service, Controller
  Status: Ishlamoqda... [▓▓▓
```

Antigravity Afzalligi	Kim Uchun Foydali
2M token context (Gemini)	Katta TS/JS loyiha — butun node_modules xotirada
Parallel agent Manager view	Ko'p komponentli loyiha — backend+frontend bir vaqtda
Brauzer agent	UI testlash — screenshot ko'rib, xatoni o'zi tuzatadi
antigravity-claude-proxy	Bepul Claude modelini Claude Code CLI'da ishlatish

DIQQAT

Antigravity hali public preview — production loyihalarida ISHLATMANG. Tez-tez xatolar, sekin generatsiya, jamoaviy funksiyalar yo'q. 2026 yil oxiriga qadar barqarorlashishi kutilmoqda.

17.6 Antigravity Proxy — Bepul Claude Code Ishlatish

Google Antigravity orqali Claude modellarini **bepul** ishlatish mumkin — maxsus proxy server orqali:

antigravity-claude-proxy — sozlash

```
# 1. O'rnatish:
npx antigravity-claude-proxy@latest start

# 2. Google akkauntni ulash:
antigravity-claude-proxy accounts add
# Brauzer ochiladi -> Google'ga login -> tayyor

# 3. Claude Code'ni proxy orqali ishga tushirish:
export ANTHROPIC_BASE_URL=http://localhost:8080
export ANTHROPIC_AUTH_TOKEN=test
claude

# Yoki alias sifatida (~/.zshrc ga):
alias claude-free='ANTHROPIC_BASE_URL=http://localhost:8080 \
  ANTHROPIC_AUTH_TOKEN=test claude'

# Model tanlash (Claude Code ichida):
> /model
# claude-opus-4-6-thinking, claude-sonnet-4-5-thinking mavjud
```

↑ *Google One AI Pro obunangiz bilan yoki bepul (cheklovlar bilan) Claude modellarini ishlatish*

ESLATMA

Google One AI Pro obunasi ($20/oy) bor bo'lsa, bu proxy orqali Claude Opus'ni ham ko'proq limit bilan ishlatish mumkin. Jio va ba'zi talaba obunalari bepul kirish beradi.

17.7 Qaysi Birini Tanlash Kerak?

Muammo bo'yicha tanlang

Vazifa / Holat	Tavsiya	Sabab
Murakkab bug (NPE, N+1, race condition)	Claude Code	200K haqiqiy context, 80.9% SWE-bench, chuqur fikrlash
Katta codebase refactoring (12M+ qator)	Claude Code Opus	Yagona to'liq context — boshqa tool yenga olmaydi
Kunlik yangi endpoint, komponent	Cursor	Tab autocomplete, tez iteratsiya, vizual diff
Yangi boshlovchi, IDE'dan chiqmaslik	Cursor (Composer)	Qulay, tanish, natija yaxshi
Bepul sinab ko'rish, tajriba	Antigravity	Bepul, Gemini 3 Pro kuchli
Katta TS loyiha, node_modules kontekst	Antigravity (Gemini)	2M token — tengsiz
CI/CD pipeline, GitHub Actions	Claude Code headless	claude -p komandasi, JSON output
Parallel frontend+backend bir vaqtda	Antigravity Manager	3 agent parallel — 2.5x tezroq
Java Spring murakkab arxitektura	Claude Code Opus	Chuqur fikrlash, annotatsiya tushunish
Vue 3 / React tezkor prototip	Cursor	Tez iteratsiya, inline suggest

17.8 Professional Dasturchi Uchun Optimal Stack

TAVSIYA ETILGAN KOMBINATSIYA (2026)

1. CURSOR (kunlik asosiy tool) $20/oy
   -> Barcha oddiy va o'rta vazifalar

-> Tab autocomplete, tezkor Composer
-> Yangi feature, refactor, debug

2. CLAUDE CODE Pro (og'ir muammolar) $20/oy
-> Boshqa tool yutqa bergan vazifalar
-> Murakkab arxitektura, 200K context
-> CI/CD, GitHub Actions, headless

3. ANTIGRAVITY (bepul bonus) $0
-> Eksperimental workflow sinash
-> Parallel agent o'rganish
-> Gemini 3 Pro bilan katta TS loyiha

Jami: $40/oy — professional darajada to'liq qoplangan

Byudjet cheklangan bo'lsa

Byudjet	Stack	Qoplanishi
$0/oy	Antigravity (Gemini 3 Pro bepul) + claude-proxy	80% qoplanadi — ko'p vazifalar uchun yetarli
$20/oy	Claude Code Pro (Sonnet 4.6, 79.6%)	95% qoplanadi — professional daraja
$40/oy	Claude Code Pro + Cursor Pro	100% — hamma holat uchun tayyor
$100/oy	Claude Code Max 5x + Cursor	Cheksiz, enterprise daraja

Optimal kunlik workflow

```
# Ertalab — Cursor bilan boshlang:
cursor .   # VS Code muhitida ochish
# Tab autocomplete, tezkor feature ishlab chiqish

# Qiyin muammo chiqqanda — Claude Code'ga o'ting:
cd ./mening-loyiham && claude
> /plan
> @src/service/OrderService.java 84-qatordagi
  murakkab N+1 query muammosini tuzat...

# CI/CD'da — headless rejimda:
git diff main...feature | claude -p \
  'Bu PR ni tekshir, xatolarni JSON da ber' \
  --output-format json
```

```
# Parallel katta vazifa — Antigravity:
# Manager View -> 3 agent -> backend+frontend+test
```

↑ *Kunlik ish oqimi: Cursor (tezkor) + Claude Code (chuqur) + Antigravity (parallel)*

17.9 Muhim Ogohlantirishlar

Muammo	Yechim
Cursor'da Claude Opus = qimmat kredit	Cursor'da 'Auto' modeli ishlating — kredit tejaydi. Opus faqat kerak bo'lganda.
Claude Code limit 5 soatlik rolling window	claude.ai web va Claude Code UMUMIY limit. Ikkalasini bir vaqtda ishlatmang.
Antigravity rate limit tez tugaydi	Bir necha Google akkaunt qo'shing, antigravity-proxy accounts load-balance qiladi.
Cursor'da '200K context' aslida 70-120K	Max Mode yoqing — lekin sekinroq va qimmatroq. Katta vazifalar uchun Claude Code ishlating.
Antigravity Claude modeli sekin (8-12s)	Antigravity'da Gemini 3 Flash ishlatgan tezroq (2-3s). Claude uchun Claude Code ishlating.